

Smart Resume Screening

Technical Documentation

AI-Powered Resume Screening System

Gemini 2.5 Flash Lite • SentenceTransformers • Cosine Similarity

Version	1.0
Date	May 14, 2026
Stack	Python 3.x, Gemini API, SentenceTransformers
Audience	Developers, ML Engineers, Technical Reviewers

Table of Contents

1. Project Overview
2. Architecture & Project Structure
 - 2.1 Module Dependency Map
 - 2.2 File Descriptions
3. Data Models (Schemas)
 - 3.1 ResumeData
 - 3.2 JDData
4. Module Reference
 - 4.1 text_extractor.py
 - 4.2 jd_parser.py
 - 4.3 resume_parser.py
 - 4.4 embeddings.py
 - 4.5 matcher.py
 - 4.6 main.py
5. Scoring Algorithm
 - 5.1 Component Weights
 - 5.2 Penalty System
 - 5.3 Verdict Thresholds
6. Setup & Configuration
 - 6.1 Prerequisites
 - 6.2 Installation
 - 6.3 Environment Variables
 - 6.4 Running the Pipeline
7. Output Format
8. Extending the System
9. Known Limitations & Future Work

1. Project Overview

Smart Resume Screening is an end-to-end AI pipeline that automatically evaluates candidate resumes against a Job Description (JD) and produces a ranked match score. It replaces manual screening by combining large-language-model (LLM) parsing with semantic vector similarity, giving recruiters an objective, explainable ranking in seconds.

Key Capabilities

- Parse JD PDFs into structured hiring criteria
 - Parse candidate resume PDFs into structured profiles
 - Embed all text fields with SentenceTransformers
 - Score each resume across 7 weighted dimensions
- Apply a missing-skills penalty for accuracy
 - Process multiple resumes in parallel (ThreadPoolExecutor)
 - Output ranked JSON leaderboard with full match details
 - Cache JD embeddings to avoid redundant API calls

Pipeline at a Glance

Step 1	PDF Extraction	PyPDFLoader reads raw text from JD and resume PDFs
Step 2	LLM Parsing	Gemini 2.5 Flash Lite converts raw text to structured JSON
Step 3	Embedding	all-MiniLM-L6-v2 encodes skills, roles, projects into vectors
Step 4	JD Cache	JD embeddings are computed once and reused for all resumes
Step 5	Matching	Cosine similarity + exact-match boost per requirement
Step 6	Scoring	Weighted formula across 7 dimensions → 0-100 score
Step 7	Output	Sorted JSON results saved to outputs/ directory

2. Architecture & Project Structure

2.1 Module Dependency Map

The diagram below shows how modules depend on each other. `main.py` is the orchestrator; all heavy work is delegated to specialised modules.

```
main.py
├── text_extractor.py (PDF → raw text)
├── jd_parser.py      (raw text → JDData JSON via Gemini)
│   ├── schemas.py   (JDData Pydantic model)
├── resume_parser.py (raw text → ResumeData JSON via Gemini)
│   ├── schemas.py   (ResumeData Pydantic model)
├── matcher.py        (score resume against cached JD)
│   └── embeddings.py (SentenceTransformer wrapper)
```

2.2 File Descriptions

main.py	Entry point. Orchestrates JD processing, parallel resume processing, ranking, and JSON output.
text_extractor.py	Wraps LangChain's PyPDFLoader to extract plain text from any PDF file.
jd_parser.py	Sends JD text to Gemini with a strict JSON prompt; validates output against JDData schema.
resume_parser.py	Sends resume text to Gemini; post-processes skills (dedupe, lowercase) and projects; validates against ResumeData.
embeddings.py	Loads all-MiniLM-L6-v2 once (lru_cache); exposes get_embedding(), get_embeddings_batch(), cosine_similarity().
matcher.py	Core scoring engine: builds resume evidence, embeds it, matches against cached JD vectors, computes weighted final score.
schemas.py	Pydantic v2 models — ResumeData and JDData — used for type-safe validation of LLM output.
.env / .env.example	Environment variable file. Must contain GEMINI_API_KEY.
requirements.txt	Pinned Python dependencies for reproducible installs.
outputs/	Auto-created directory for timestamped JSON result files.
resumes/	Default directory for candidate PDF resumes and the JD PDF.

3. Data Models (Schemas)

3.1 ResumeData

Defined in schemas.py. Populated by resume_parser.py.

Field	Type	Description
candidate_name	<i>str</i>	Full name extracted from the resume.
email	<i>str</i>	Email address (only if clearly visible).
phone	<i>str</i>	Phone number (only if clearly visible).
skills	<i>List[str]</i>	Deduplicated, lowercased technical and professional skills.
experience_years	<i>float</i>	Total professional experience in years (numeric).
companies	<i>List[str]</i>	Companies the candidate has worked at.
education	<i>str</i>	Highest qualification only.
projects	<i>List[Project]</i>	Structured project objects (see below).
roles	<i>List[str]</i>	Job titles / designations held.

Project sub-model fields:

title	<i>str</i>	Project name.
description	<i>str</i>	Concise semantic summary of the project.
highlights	<i>List[str]</i>	Short achievement chunks (e.g. 'reduced hallucination 60%→15%').
technologies	<i>List[str]</i>	Frameworks, libraries, cloud services used (lowercase, deduplicated).

3.2 JDData

Defined in schemas.py. Populated by jd_parser.py.

required_skills	<i>List[str]</i>	Must-have skills explicitly stated in the JD (lowercase).
preferred_skills	<i>List[str]</i>	Nice-to-have skills (lowercase).
required_experience	<i>float</i>	Minimum years of experience required (uses range minimum).

job_role	str	Job title in lowercase (e.g. 'machine learning engineer intern').
education_required	str	Required degree as a short string.
responsibilities	List[str]	Key job responsibilities (concise bullet points).

4. Module Reference

4.1 text_extractor.py

Provides a single utility function that wraps LangChain's PyPDFLoader to extract plain text from any PDF file path.

```
def extract_text_from_pdf(file_path: str) -> str:
    """
    Raises FileNotFoundError if the path does not exist.
    Raises RuntimeError if PDF parsing fails.
    Returns stripped plain text string.
    """
```

Dependencies: langchain_community, PyPDFLoader. Page content from all pages is joined with double newlines.

4.2 jd_parser.py

Calls Gemini 2.5 Flash Lite with a detailed zero-shot prompt to extract structured hiring requirements. The response is strictly validated against JDData.

```
def parse_jd(jd_text: str) -> dict:
    # 1. Build structured prompt with schema instructions
    # 2. Call Gemini (temperature=0, top_p=0.1 for determinism)
    # 3. Strip markdown fences if present
    # 4. json.loads() -> JDData(**data).model_dump()
    # 5. On error: return safe empty JDData fallback
```

Error handling: JSONDecodeError and all other exceptions are caught; a safe empty JDData object is returned so the pipeline never crashes.

4.3 resume_parser.py

The most complex parser. Extracts detailed candidate information including projects with highlights and technologies. Includes several helper functions for post-processing.

extract_first_json(text)	Robustly extracts the first valid JSON object using raw_decode(), ignoring any surrounding text.
clean_json_response(text)	Strips markdown code fences (``json ... ``) from Gemini responses.
deduplicate_skills(skills)	Lowercases and deduplicates a skill list, removing non-string values.
clean_projects(projects)	Normalises each project dict: validates types, deduplicates technologies, strips whitespace.

4.4 embeddings.py

Thin wrapper around `SentenceTransformer('all-MiniLM-L6-v2')`. The model is loaded exactly once via `lru_cache(maxsize=1)` — thread-safe and memory-efficient.

<code>get_model()</code>	Returns (and caches) the <code>SentenceTransformer</code> model instance.
<code>get_embedding(text)</code>	Embeds a single string; returns a normalised numpy array (384-dim).
<code>get_embeddings_batch(texts)</code>	Embeds a list of strings in a single forward pass (<code>batch_size=64</code>). Returns list of arrays.
<code>cosine_similarity(a, b)</code>	Returns float similarity in <code>[0, 1]</code> . Handles zero-norm vectors gracefully.

4.5 matcher.py

The core engine. Contains all scoring logic and the JD preprocessing cache.

normalize_skill(skill)	Lowercases, normalises spaces, removes punctuation variations, strips brackets.
enrich(text, role)	Wraps text in context (e.g. 'Professional concept: python. Relevant for machine learning engineer intern.') to improve embedding quality.
preprocess_jd(jd)	Embeds all JD fields (skills, responsibilities, role, education) in one batched call and stores vectors in a dict.
get_cached_jd(jd_id, jd)	Checks jd_cache dict; calls preprocess_jd() only on cache miss. Uses MD5 of JD text as key.
build_resume_evidence(resume)	Converts resume dict into a flat list of evidence dicts (type + text) covering skills, roles, project descriptions, highlights, and technologies.
embed_resume_evidence(evidence, role)	Batch-embeds all evidence texts with role context; attaches embedding arrays in-place.
match_requirement(req, vec, evidence, threshold)	Finds the highest cosine-similarity evidence for a requirement. Applies exact-match boost (score → 0.98).
education_match(edu, edu_req)	Normalises both strings (e.g. btech → 'bachelor degree') then computes cosine similarity.
experience_score(cand, req)	Returns (min(cand/req, 1.0), gap_years). Returns (1.0, 0) when no experience is required.
global_similarity(evidence, jd_cached)	Computes holistic similarity: concatenates all evidence text vs. all JD text; single cosine similarity.
match_resume_to_jd(resume, jd_cached)	Orchestrates all sub-scores, applies penalty, returns full result dict.

4.6 main.py

The entry point and batch orchestrator.

generate_jd_id(jd_text)	MD5 hash of JD text — used as cache key.
collect_resume_paths(folder, fallback)	Returns sorted list of .pdf paths from RESUME_FOLDER; falls back to SINGLE_RESUME_PATH.
process_jd(jd_path)	Full JD pipeline: extract text → Gemini parse → embed and cache. Runs once.

<code>process_single_resume(path, jd_cached)</code>	Per-resume worker: extract → parse → match. Returns result dict; never raises.
<code>process_resumes_parallel(paths, jd_cached, max_workers)</code>	ThreadPoolExecutor with up to MAX_WORKERS (default 10) threads. Collects results as futures complete; sorts by score descending.
<code>save_results(results, filename)</code>	Serialises results to outputs/match_results_.json.
<code>main(jd_path, resume_paths, max_workers)</code>	Top-level function: calls all steps, prints ranked leaderboard, saves JSON.

5. Scoring Algorithm

5.1 Component Weights

The final score is a weighted sum of seven independent sub-scores, each normalised to [0, 1], then multiplied by a penalty factor and scaled to 0–100.

Required Skill Score (skill_score)		35%
<i>Avg cosine similarity of required JD skills vs. best resume evidence</i>		
Preferred Skill Score (pref_score)		15%
<i>Avg cosine similarity of preferred JD skills</i>		
Project/Responsibility Score		15%
<i>Avg match of JD responsibilities vs. project descriptions & highlights</i>		
Role Score (role_score)		10%
<i>Max cosine sim of resume roles/projects vs. JD job role vector</i>		
Experience Score (exp_score)		10%
<i>min(candidate_exp / required_exp, 1.0)</i>		
Education Score (edu_score)		5%
<i>Cosine similarity of normalised education strings</i>		
Global Score (global_score)		10%
<i>Holistic cosine sim of full resume text vs. full JD text</i>		

Formula:

```
final = (0.35 * skill_score
        + 0.15 * pref_score
        + 0.15 * project_score
        + 0.10 * role_score
        + 0.10 * exp_score
        + 0.05 * edu_score
        + 0.10 * global_score)
final = final * penalty
final = round(final * 100) # → integer 0-100
```

5.2 Penalty System

A penalty multiplier is applied when required skills are missing. This prevents a candidate who scores well on project/global similarity from ranking above candidates who actually have the must-have skills.

```
missing_ratio = len(unmatched_required) / max(1, len(required_skills))
penalty = 1 - (missing_ratio * 0.25)
penalty = max(0.75, penalty) # floor at 0.75 (never more than 25% reduction)
```

At most, the penalty reduces the final score by 25%. A candidate missing all required skills will have their score multiplied by 0.75.

5.3 Verdict Thresholds

Score Range	Verdict	Recommended Action
85 – 100	Strong Match	Prioritise for interview
70 – 84	Good Match	Schedule screening call
55 – 69	Moderate Match	Consider if pool is thin
40 – 54	Weak Match	Review manually
0 – 39	Low Match	Likely not suitable

6. Setup & Configuration

6.1 Prerequisites

- Python 3.10 or higher
- A Google Gemini API key (free tier supports ~15 req/min)
- Internet access for Gemini API calls and model downloads
- At least one JD PDF and one or more candidate resume PDFs

6.2 Installation

```
# 1. Clone / unzip the project
cd Smart_Resume_Screening

# 2. Create and activate a virtual environment (recommended)
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# requirements.txt pins:
# sentence-transformers==2.6.1
# torch==2.2.2
# numpy==1.26.4
# pydantic==2.12.5
# google-generativeai==1.66.0
# pdfplumber==0.11.9
# python-dotenv==1.2.1
# langchain-community (for PyPDFLoader)
```

6.3 Environment Variables

Copy `.env.example` to `.env` and fill in your key:

```
cp .env.example .env

# .env contents:
GEMINI_API_KEY="your_gemini_api_key_here"
```

Never commit your `.env` file. It is excluded from version control by default. The `.env.example` file is safe to commit — it contains no real keys.

6.4 Running the Pipeline

Option A — Direct execution (edit paths in `main.py`):

```
# Edit these constants at the top of main.py:
JD_PATH = r"/path/to/your/job_description.pdf"
RESUME_FOLDER = r"/path/to/your/resumes/"
MAX_WORKERS = 10    # lower for free-tier Gemini (<=15 req/min)

python main.py
```

Option B — Programmatic call (import into another service):

```
from main import main

results = main(
    jd_path="/path/to/jd.pdf",
    resume_paths=["/path/resume1.pdf", "/path/resume2.pdf"],
    max_workers=5
)

# results is a list of dicts sorted by score descending
for r in results:
    print(r['score'], r['verdict'], r['candidate_name'])
```

7. Output Format

Results are saved to `outputs/match_results_YYYYMMDD_HHMMSS.json`. The file contains a JSON array of result objects, one per resume, sorted by score descending. Below is an annotated sample of a single result object:

```
{
  "resume":      "Gaohar Imran Resume.pdf",
  "status":      "ok",                // 'ok' or 'error'
  "candidate_name": "GAOHAR IMRAN",
  "email":       "gaoharimran3@gmail.com",
  "phone":       "+91 7065284087",
  "score":       82,                  // 0-100 integer
  "verdict":     "Good Match",

  // Sub-scores (all 0.0 - 1.0 floats)
  "skill_score":    0.834,
  "preferred_skill_score": 0.761,
  "project_score":  0.712,
  "role_score":     0.691,
  "experience_score": 1.0,
  "education_score": 0.95,
  "global_score":   0.803,
  "experience_gap": 0,                // years below requirement

  // Detailed skill match breakdown
  "required_matches": [ { "requirement": "python",
                          "score": 0.98, "matched": true,
                          "evidence": "python",
                          "evidence_type": "skill" }, ... ],
  "required_missing": [ ... ],        // unmatched required skills
  "preferred_matches": [ ... ],
  "preferred_missing": [ ... ]
}
```

status	'ok' if processing succeeded, 'error' + error message if it failed.
score	Final integer score 0–100.
verdict	Human-readable tier string based on score thresholds.
skill_score ... global_score	Individual component scores (float 0–1) for debugging and transparency.
experience_gap	Years by which candidate falls short of required experience (0 if met).
required_matches / required_missing	Per-skill match detail: which evidence matched and at what score.
preferred_matches / preferred_missing	Same structure for preferred skills.

8. Extending the System

Swap the Embedding Model

Replace 'all-MiniLM-L6-v2' in `embeddings.py` with any SentenceTransformer model. Larger models (e.g. all-mpnet-base-v2) improve accuracy at the cost of speed. Update the zero vector dimension (currently 384) in `matcher.py` if needed.

Change the LLM

Replace the Gemini client in `jd_parser.py` and `resume_parser.py` with any API-compatible provider (OpenAI, Anthropic, Mistral). Keep the prompt and schema validation logic identical.

Adjust Score Weights

Edit the weighted formula at the bottom of `match_resume_to_jd()` in `matcher.py`. Ensure weights sum to 1.0.

Tune Match Thresholds

The cosine similarity threshold for 'matched: true' is set per-call in `match_resume_to_jd()`: 0.68 for required skills, 0.63 for preferred, 0.60 for responsibilities. Adjust to taste.

Add a Web / REST API

Wrap `main()` or `process_single_resume()` in a FastAPI endpoint. Accept PDF uploads, call the pipeline, and return the result dict as JSON.

Persist Results to a Database

Replace `save_results()` with a database write (PostgreSQL, MongoDB). The result dict maps cleanly to a document or row.

Support Non-PDF Inputs

Replace PyPDFLoader in `text_extractor.py` with python-docx or other parsers for Word documents or plain text files.

Add a UI

Build a Streamlit or React front-end that calls `main()` and renders the ranked leaderboard, skill match charts, and per-candidate breakdowns visually.

9. Known Limitations & Future Work

Known Limitations

Gemini Rate Limits

The free tier of Gemini allows ~15 requests/minute. With `MAX_WORKERS=10` and both JD+resume parse calls, bursts of >7 resumes may trigger 429 errors. Reduce `MAX_WORKERS` or add exponential backoff retries.

Hardcoded File Paths

`JD_PATH`, `RESUME_FOLDER`, and `SINGLE_RESUME_PATH` are hardcoded constants in `main.py`. For production use, these should be passed as CLI arguments or environment variables.

Education Normalisation

The `normalize_education()` function only handles a few common degree patterns (btech, bachelor, mtech, master). Unusual degree names may not normalise correctly.

No Persistent Embedding Cache

The JD cache (`jd_cache` dict) is in-memory and lost on restart. For long-running services, persist embeddings to disk (numpy `.npy` files or a vector DB).

PDF Quality Dependence

Text extraction quality depends on the PDF. Scanned image-only PDFs will produce empty text and fail silently. Add OCR (pytesseract / pdf2image) for scanned documents.

English-Only

Prompts and the embedding model are optimised for English. Multi-language resumes may produce degraded results.

Suggested Future Improvements

- Add CLI argument parsing (`argparse` / `typer`) for `jd_path`, `resume_folder`, `max_workers`.
- Implement retry logic with exponential backoff for Gemini API calls.
- Persist JD embeddings to disk using `numpy.save()` for faster restarts.
- Add OCR support for scanned PDF resumes using `pytesseract` + `pdf2image`.
- Build a Streamlit dashboard for visual score breakdowns and candidate comparison.
- Add unit tests for parsers (mock Gemini responses), scorer, and text extractor.
- Support batch JD processing — score one pool of resumes against multiple JDs.
- Integrate with ATS platforms (Greenhouse, Lever) via REST API.

- Add multilingual support via multilingual SentenceTransformer models.
- Store results in a vector DB (Pinecone, Chroma) for historical search and analytics.

Documentation generated on May 14, 2026 • Smart Resume Screening v1.0